INGENIERÍA DE SOFTWARE I / 501T SIS **RETO 01**

ING. WILLIAM ALEXANDER MATALLANA PORRAS

UNIEVRSIDAD DE CUNDINAMARCA

RETO DE INGENIERÍA DE SOFTWARE

AlimentaRed

Plataforma de rescate y distribución responsable de alimentos

GUÍA DE TRABAJO COLABORATIVO | IMPACTO SOCIAL

PROPÓSITO DE LA ACTIVIDAD

Diseñar, documentar, construir y demostrar una solución digital que responda al problema planteado. La evaluación considerará el proceso de ingeniería, la coherencia entre artefactos y la funcionalidad real del backend y de la aplicación móvil. Las propuestas se compararán con una rúbrica común.

1. Presentación del reto y necesidad del cliente

En una ciudad, restaurantes, cafeterías y comercios pueden disponer de alimentos aptos para donar, mientras organizaciones comunitarias requieren apoyo. Actualmente, publicar, solicitar, asignar y registrar entregas depende de mensajes informales y dificulta comprobar quién recibió qué y cuándo.

Solicitud del cliente simulado: Una red comunitaria quiere una solución móvil que permita a comercios publicar lotes de alimentos, a organizaciones solicitar o reservar donaciones y a coordinadores verificar asignaciones y entregas. Necesita evitar reservas duplicadas, controlar fechas de consumo seguro y consultar indicadores de alimentos recuperados.

2. Propósito, objetivos y alcance

Objetivo general: diseñar e implementar una solución móvil conectada a una API REST, aplicando las cuatro fases de RUP y evidenciando los requisitos y reglas que gobiernan el proceso principal.

- Definir problema, interesados, actores, alcance, restricciones e indicadores de éxito.
- Modelar los procesos y los datos antes de codificar; mantener consistencia entre documentación e implementación.
- Implementar autenticación, operaciones CRUD, proceso transaccional central, consultas e indicadores.
- Construir una app Android funcional que consuma los endpoints del servidor y permita una demostración de extremo a extremo.

Incluido: prototipo funcional con datos de prueba, documentación y validación técnica. Fuera del alcance obligatorio: integraciones con entidades reales, pagos, infraestructura productiva y tratamiento de información privada real. Servicios externos pueden ser simulados y deben indicarse como tales.

3. Actores y modelo inicial del dominio

Actor	Responsabilidad esperada
Donante	Publica lotes, actualiza disponibilidad y confirma la entrega.
Organización receptora	Consulta ofertas, solicita lotes y confirma recepción.
Coordinador	Revisa solicitudes, asignaciones, incidencias e indicadores.
Administrador	Gestiona usuarios, roles y parámetros generales.

Entidades sugeridas (no son un modelo definitivo): Usuario / Rol, Organización, LoteAlimento, CategoriaAlimento, SolicitudDonacion, Asignacion, Entrega, Incidencia.

Cada equipo deberá definir atributos, identificadores, relaciones, cardinalidades, restricciones, estados, clases de dominio y por lo menos una justificación de diseño distinta a la lista sugerida.

4. Requisitos funcionales mínimos

Los siguientes requisitos deben analizarse y descomponerse cuando corresponda. Para cada uno documentar actor, prioridad, precondiciones, criterios de aceptación, excepciones y vínculo con casos de uso, endpoints y pruebas.

ID	Capacidad que debe implementar el sistema
RF01	Registrar e iniciar sesión de manera segura y mostrar opciones según el rol.
RF02	Registrar y consultar organizaciones donantes y receptoras.
RF03	Publicar un lote con nombre, categoría, cantidad, unidad, fecha límite y estado.
RF04	Consultar y filtrar lotes por categoría, ubicación general, estado y fecha.
RF05	Crear una solicitud de donación para un lote disponible.
RF06	Aceptar o rechazar solicitudes y asignar un lote a una organización receptora.
RF07	Registrar el retiro y la recepción, con fecha, responsable y observaciones.
RF08	Cancelar solicitudes o reportar incidencias con trazabilidad de estados.
RF09	Consultar el historial de donaciones y las estadísticas por periodo y categoría.

5. Reglas de negocio y atributos de calidad

Reglas iniciales que deben convertirse en validaciones, cambios de estado y pruebas:

- No se puede solicitar un lote que esté vencido, cancelado o entregado.
- La cantidad asignada no puede superar la cantidad disponible del lote.
- Una asignación aprobada debe disminuir la disponibilidad de forma consistente.
- Solo el donante responsable o un coordinador autorizado puede confirmar la salida.
- Cada cambio de estado debe conservar fecha y usuario responsable.
- Las condiciones de inocuidad se definen como requisitos operativos; la app no certifica que un alimento sea seguro.

Identifique y justifique al menos cuatro reglas adicionales propias del proyecto. Establezca requisitos no funcionales verificables para seguridad, rendimiento, disponibilidad ante fallos, integridad, mantenibilidad y usabilidad (con métrica o evidencia concreta).

6. Proceso de desarrollo obligatorio: RUP

Se aplicará RUP de manera iterativa y basada en casos de uso. No basta con mencionar las fases: cada equipo entregará evidencias fechadas, responsables, resultados y decisiones. Las iteraciones pueden adecuarse al cronograma del curso.

Fase RUP	Trabajo del equipo	Evidencia verificable
Inicio	Problema, actores, visión, alcance, factibilidad y riesgos iniciales.	Visión del proyecto; glosario; lista de interesados; casos de uso prioritarios; plan preliminar.
Elaboración	Precisar reglas, arquitectura, modelos y riesgos; prototipar flujos.	Especificación de requisitos; diagramas UML; modelo relacional; mockups;

		arquitectura; matriz de riesgos.
Construcción	Implementar API, persistencia y app móvil en incrementos; integrar y probar.	Repositorio Git; endpoints documentados; pantallas funcionales; pruebas y evidencias por iteración.
Transición	Validar con cliente simulado, corregir defectos y entregar versión ejecutable.	Acta de aceptación simulada; manual breve; README; APK o proyecto Android; video y sustentación.

Por cada iteración: objetivo, casos de uso abordados, tareas y responsables, resultado demostrable, defectos encontrados y ajustes. Registrar al menos una solicitud de cambio simulada y su impacto en requisitos, modelos, código y pruebas.

7. Diagramas y trazabilidad

Artefacto	**Condición mínima**
Casos de uso	Actores, frontera, relaciones pertinentes; descripción completa de los casos críticos.
Clases	Clases del dominio, atributos, métodos relevantes, asociaciones y multiplicidades.
Actividades	Proceso de negocio principal, decisiones, excepciones y estados.
Secuencia	Interacción app → ViewModel/repositorio → API → servicio → repositorio → base de datos; caso de error.
Modelo relacional	Tablas, PK, FK, normalización y diccionario de datos.
Componentes / arquitectura	Arquitectura lógica de backend por capas, cliente Android MVVM y dependencias.
Despliegue	App, servidor API, base de datos, conexiones, protocolos y configuración de pruebas.

Matriz de trazabilidad obligatoria: ID del requisito → caso de uso → pantalla → endpoint → entidad/clase → prueba → evidencia. Si un requisito cambia, actualizar los artefactos asociados.

8. Aplicación móvil: pantallas y experiencia de usuario

Diseñar mockups antes de implementar. La aplicación debe usar Kotlin, Jetpack Compose, Navigation, ViewModel, Retrofit (o cliente HTTP equivalente) y manejo de estados de pantalla. Las operaciones importantes deben consumir datos reales de la API, no listas fijas.

Pantallas mínimas del proyecto	**Criterio de implementación**
Inicio de sesión / registro	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Inicio con lotes disponibles y filtros	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Formulario de publicación de donación	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Detalle del lote y solicitud	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Mis solicitudes / mis donaciones	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Seguimiento y confirmación de entrega	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.
Panel de indicadores por rol	Mostrar interacción real, validación, estado vacío/carga/error y navegación según rol.

Los diseños deben ser coherentes con los roles, entidades, reglas de negocio y respuestas del servidor. Incluir mensajes de éxito y error, confirmación de acciones críticas y presentación visual uniforme.

9. Backend, arquitectura y endpoints

Backend obligatorio: Java con Spring Boot, Spring Web, Spring Data JPA, postgreSQL, validaciones, manejo global de excepciones, OpenAPI/Swagger y separación por paquetes config, controller, service, repository, entity, dto y exception. Implementar autenticación y autorización por roles; nunca almacenar contraseñas en texto plano.

Método	Ruta orientativa	Propósito
GET	/api/lotes	Listar y filtrar lotes
POST	/api/lotes	Crear lote
GET	/api/lotes/{id}	Ver detalle
POST	/api/solicitudes	Solicitar lote
PATCH	/api/solicitudes/{id}/resolver	Aprobar o rechazar
PATCH	/api/entregas/{id}/confirmar	Confirmar recepción
GET	/api/reportes/impacto	Consultar indicadores

Agregar los endpoints transversales: POST /api/auth/login, POST /api/auth/register (si aplica), GET /api/usuarios/me, GET /api/health, /swagger-ui/index.html y /v3/api-docs. Las rutas son orientativas: se permiten cambios justificados si se mantienen capacidades equivalentes.

Documentar cada operación: método, ruta, parámetros, DTO de solicitud y respuesta, errores HTTP, permisos y evidencia de ejecución desde Swagger. El README debe incluir instalación, variables de entorno, script de base de datos y usuarios ficticios de prueba.

10. Validación, pruebas y demostración

Ejecución funcional obligatoria para la sustentación: Un donante publica un lote, una organización lo solicita, un coordinador aprueba y se confirma una entrega sin que el lote pueda asignarse por encima de su disponibilidad.

- Pruebas de registro/autenticación y denegación de permisos.
- Pruebas de CRUD y consultas con filtros; verificar persistencia real.
- Pruebas del flujo central, transiciones de estado y al menos dos reglas de negocio.
- Pruebas con datos inválidos, registros inexistentes y fallos de red en la app.
- Mínimo diez pruebas automatizadas en backend, con resultados reproducibles.
- Evidencia de al menos una operación completa ejecutada desde la aplicación móvil contra la API.

11. Riesgos, planificación y trabajo colaborativo

Construir una matriz con riesgo, causa, probabilidad, impacto, responsable, mitigación y seguimiento. Considerar disponibilidad del equipo, errores de modelado, integración móvil–API, seguridad, cambios en los requisitos y cumplimiento del cronograma.

Todos los integrantes deberán intervenir en la explicación del sistema. El equipo organizará las responsabilidades de requisitos/RUP, UML/datos, backend, aplicación móvil y pruebas/integración; las contribuciones deben evidenciarse mediante commits e historial de tareas.

12. Entrega final

Entregable	Contenido exigido
Informe académico	Portada, resumen, contexto, objetivos, alcance, resultados por fase RUP, requisitos, decisiones de arquitectura, riesgos, pruebas, conclusiones y bibliografía APA 7.
Anexos técnicos	Diagramas legibles/editables, matriz de trazabilidad, diccionario de datos, mockups y evidencias.
Backend	Repositorio, código, script o migraciones de MySQL, Swagger funcional, pruebas y README.
Android	Código fuente Compose/MVVM, pantallas y flujos integrados, instrucciones para ejecución y APK si es viable.
Sustentación	Demo en vivo de extremo a extremo, participación del equipo y defensa de decisiones.